来自 CLAUDE CODE 的经验

提示缓存决定一切

稳定前缀如何让长时间运行的智能体更快、更省钱、更可扩展。

缓存支配智能体的一切

只有重复上下文足够便宜，长时间运行的智能体才真正可行。

Claude Code 像对待生产可靠性一样对待缓存命中率：一旦下降，就触发告警和事故处理。

更低
每轮延迟

更低
输入成本

更高
速率限制余量

提示缓存本质上是前缀匹配

共享前缀

从请求开头到每个缓存断点之间的内容，都必须逐字节完全一致。

默认很脆弱

时间戳、工具顺序变化，或工具结构定义改动，都可能让缓存前缀失效。

为复用而组织提示

1. 静态系统提示与工具
2. 项目指令
3. 会话上下文
4. 持续增长的对话

静态内容在前，动态内容在后。

这种顺序能让许多会话共享早期缓存命中，只让实时对话部分发生变化。

用消息承载更新

破坏缓存的直觉做法

- 修改系统提示
- 插入新的时间戳
- 就地重写指令

缓存安全的模式

- 在下一轮添加消息
- 使用聚焦的系统提醒
- 保留稳定前缀

> 变化的信息应该放在末尾附近，而不是塞进已缓存的开头。

不要在会话中途切换模型

100k

用 Opus 缓存的词元，继续使用可能比为 Haiku 重建缓存更便宜。

更好的出口

父模型	准备交接信息
子智能体	带着聚焦上下文启动
主会话	保持缓存完整

不要在会话中途增删工具

工具也是前缀

改变工具集合会让整段对话的缓存失效。

计划模式

保留所有工具；用工具驱动的状态加消息来表达计划模式。

工具搜索

使用稳定的轻量级桩并延迟加载，而不是移除工具。

压缩里藏着成本陷阱

单独调用摘要器

不同的系统提示。没有工具。前缀立刻分叉。

完整输入不命中缓存

缓存安全的分叉

相同的系统提示、上下文、工具和历史；压缩提示放在最后。

只新增词元

预留压缩缓冲区，确保提示和摘要输出仍能放得下。

像运营基础设施一样运营缓存

度量
缓存命中率、延迟和输入开销。

告警
前缀被破坏，值得按生产问题处理。

设计
功能默认必须保留稳定前缀。

评审
发布前检查工具结构定义、顺序和模型选择。

智能体设计检查清单

1. 把提示缓存当作前缀约束。
2. 稳定材料放前面，实时更新放最后。
3. 保持模型和工具定义稳定。
4. 在保留父前缀的前提下分叉工作。
5. 像监控可用性一样监控缓存健康。

从第一天起围绕缓存设计，而不是等成本飙升后再补救。

来源：Anthropic，《构建 Claude Code 的经验：提示缓存决定一切》，2026 年 4 月 30 日。